AI ASSISTED CODING

Name: P.Swaran Raj **Roll.No:** 2303A52247

Batch: 37 **Assignment -** 19.3

Task 1 – Python to C++ Conversion

Prompt:

"# Convert a Python Student class with attributes (id, name, marks) and a display method into equivalent C++ code with constructor and proper access specifiers."

```python
class Student:
    def __init__(self, id, name, marks):
        self.id = id
        self.name = name
        self.marks = marks

    def display(self):
        print(f"Student ID: {self.id}")
        print(f"Student Name: {self.name}")
        print(f"Student Marks: {self.marks}")

# Example usage in Python
student1 = Student(101, "Alice", 85)
student1.display()
```

```cpp
# include <iostream>
# include <string>

class Student {
private:
    int id;
    std::string name;
    int marks;

public:
    // Constructor
    Student(int studentId, std::string studentName, int studentMarks) {
        id = studentId;
        name = studentName;
        marks = studentMarks;
    }

    // Display method
    void display() {
        std::cout << "Student ID: " << id << std::endl;
        std::cout << "Student Name: " << name << std::endl;
        std::cout << "Student Marks: " << marks << std::endl;
    }
};

int main() {
    // Create a Student object
    Student student1(101, "Alice", 85);

    // Call the display method
    student1.display();
```

```
    std::cout << "\n";

    Student student2(102, "Bob", 92);
    student2.display();

    return 0;
}
```

```
%%writefile student.cpp
#include <iostream>
#include <string>

class Student {
private:
    int id;
    std::string name;
    int marks;

public:
    // Constructor
    Student(int studentId, std::string studentName, int studentMarks) {
        id = studentId;
        name = studentName;
        marks = studentMarks;
    }

    // Display method
    void display() {
        std::cout << "Student ID: " << id << std::endl;
        std::cout << "Student Name: " << name << std::endl;
        std::cout << "Student Marks: " << marks << std::endl;
    }
};
```

```cpp
int main() {
    // Create a Student object
    Student student1(101, "Alice", 85);

    // Call the display method
    student1.display();

    std::cout << "\n";

    Student student2(102, "Bob", 92);
    student2.display();

    return 0;
}
```

```
Writing student.cpp
```

```
!g++ student.cpp -o student
!./student
```

```
Student ID: 101
Student Name: Alice
Student Marks: 85

Student ID: 102
Student Name: Bob
Student Marks: 92
```

Explanation:

The AI translated a Python class into C++ by adapting syntax, data types, and structure. It introduced constructors and access specifiers like public and private, which are required in C++. The functionality of the class

remained the same in both languages. This task shows how object-oriented concepts are implemented differently across languages.

Task 2 – Java to Python Function Conversion

Prompt:

“# Convert a Java function that checks if a number is prime into Python, ensuring correct logic and Pythonic syntax.”

```java
class PrimeChecker {
    public boolean isPrime(int n) {
        if (n <= 1) {
            return false;
        }
        for (int i = 2; i * i <= n; i++) {
            if (n % i == 0) {
                return false;
            }
        }
        return true;
    }
}

// Example usage in Java
// PrimeChecker checker = new PrimeChecker();
// System.out.println(checker.isPrime(7)); // true
// System.out.println(checker.isPrime(10)); // false
```

Output:

```python
import math

def is_prime(n):
    """
    Checks if a number is prime.
    A prime number is a natural number greater than 1 that
    has no positive divisors other than 1 and itself.
    """
    if n <= 1:
        return False
    # We only need to check for divisors up to the square root of n
    for i in range(2, int(math.sqrt(n)) + 1):
        if n % i == 0:
            return False
    return True

# Example usage in Python
print(f"Is 7 prime? {is_prime(7)}")
print(f"Is 10 prime? {is_prime(10)}")
print(f"Is 2 prime? {is_prime(2)}")
print(f"Is 1 prime? {is_prime(1)}")
print(f"Is 0 prime? {is_prime(0)}")
print(f"Is 29 prime? {is_prime(29)}")
print(f"Is 50 prime? {is_prime(50)}")
```

```
Is 7 prime? True
Is 10 prime? False
Is 2 prime? True
```


```
Is 7 prime? True
Is 10 prime? False
Is 2 prime? True
Is 1 prime? False
Is 0 prime? False
Is 29 prime? True
Is 50 prime? False
```


Explanation:

The AI converted a Java prime-checking method into Python while maintaining the same logic. It simplified syntax by removing type declarations and using Python's dynamic typing. The function was tested with multiple inputs to ensure correctness. This highlights the differences in syntax and simplicity between Java and Python.

Task 3 – Pseudocode to Python Implementation

Prompt:

"# Convert bubble sort pseudocode into Python code and test it with sample input to verify correctness."

```
procedure bubbleSort( list : array of items )

  for all elements of list do
    for i from 0 to list.length - 2 do
      if list[i] > list[i+1] then
        swap(list[i], list[i+1])
      end if
    end for
  end for

end procedure
```

```python
def bubble_sort(arr):
    n = len(arr)
    # Traverse through all array elements
    for i in range(n):
        # Last i elements are already in place
        for j in range(0, n - i - 1):
            # Traverse the array from 0 to n-i-1
            # Swap if the element found is greater than the next element
            if arr[j] > arr[j + 1]:
                arr[j], arr[j + 1] = arr[j + 1], arr[j]
    return arr

print("Bubble Sort function defined.")
```

```
Bubble Sort function defined.
```

Let's test the `bubble_sort` function with some sample input to verify its correctness.

```python
# Sample input 1
arr1 = [64, 34, 25, 12, 22, 11, 90]
print(f"Original array 1: {arr1}")
sorted_arr1 = bubble_sort(arr1.copy()) # Use .copy() to avoid modifying original list
print(f"Sorted array 1:   {sorted_arr1}")

print("\n")

# Sample input 2
arr2 = [5, 1, 4, 2, 8]
print(f"Original array 2: {arr2}")
sorted_arr2 = bubble_sort(arr2.copy())
print(f"Sorted array 2:   {sorted_arr2}")

print("\n")

# Sample input 3 (already sorted)
```

```python
# Sample input 3 (already sorted)
arr3 = [1, 2, 3, 4, 5]
print(f"Original array 3: {arr3}")
sorted_arr3 = bubble_sort(arr3.copy())
print(f"Sorted array 3:   {sorted_arr3}")

print("\n")

# Sample input 4 (reverse sorted)
arr4 = [5, 4, 3, 2, 1]
print(f"Original array 4: {arr4}")
sorted_arr4 = bubble_sort(arr4.copy())
print(f"Sorted array 4:   {sorted_arr4}")
```

```
Original array 1: [64, 34, 25, 12, 22, 11, 90]
Sorted array 1:   [11, 12, 22, 25, 34, 64, 90]


Original array 2: [5, 1, 4, 2, 8]
Sorted array 2:   [1, 2, 4, 5, 8]


Original array 3: [1, 2, 3, 4, 5]
Sorted array 3:   [1, 2, 3, 4, 5]


Original array 4: [5, 4, 3, 2, 1]
Sorted array 4:   [1, 2, 3, 4, 5]
```

Explanation:

The AI translated pseudocode into executable Python code by converting logical steps into proper syntax. The algorithm was tested using sample inputs to verify correct sorting. This demonstrates how pseudocode can be transformed into working programs. It also shows how AI helps in understanding algorithm implementation.

Task 4 – SQL to Pandas Query

Prompt:

“# Convert SQL query (SELECT name, salary FROM employees WHERE salary > 50000) into equivalent Pandas DataFrame code.”

Output:

```python
import pandas as pd

# Create a sample DataFrame to simulate the 'employees' table
data = {
    'id': [1, 2, 3, 4, 5],
    'name': ['Alice', 'Bob', 'Charlie', 'David', 'Eve'],
    'salary': [60000, 45000, 75000, 50000, 80000]
}
employees_df = pd.DataFrame(data)

print("Original DataFrame:")
display(employees_df)
```

```
Original DataFrame:
```

	id	name	salary
0	1	Alice	60000
1	2	Bob	45000
2	3	Charlie	75000
3	4	David	50000
4	5	Eve	80000

```python
# Equivalent Pandas DataFrame code for: SELECT name, salary FROM employees WHERE salary > 50000

# Select 'name' and 'salary' columns and filter rows where 'salary' is greater than 50000
result_df = employees_df[employees_df['salary'] > 50000][['name', 'salary']]

print("Resulting DataFrame (equivalent to SQL query):")
display(result_df)
```

```
Resulting DataFrame (equivalent to SQL query):
```

	name	salary
0	Alice	60000
2	Charlie	75000
4	Eve	80000

Explanation:

The AI translated a SQL query into Pandas operations using filtering and column selection. It replaced SQL conditions with DataFrame syntax in Python. The result matched the expected output from the SQL query. This shows how database queries can be handled using Python libraries.

Task 5 – Real-Time Application: Algorithm Translation Across

Prompt:

“# Translate a Python searching or sorting algorithm into another language (C or JavaScript) and explain syntax differences and challenges.”

Output:

```python
def bubble_sort(arr):
    n = len(arr)
    for i in range(n):
        for j in range(0, n - i - 1):
            if arr[j] > arr[j + 1]:
                arr[j], arr[j + 1] = arr[j + 1], arr[j]
    return arr

# Example usage in Python
python_arr = [64, 34, 25, 12, 22, 11, 90]
print(f"Original Python array: {python_arr}")
sorted_python_arr = bubble_sort(python_arr.copy())
print(f"Sorted Python array:   {sorted_python_arr}")
```

```
Original Python array: [64, 34, 25, 12, 22, 11, 90]
Sorted Python array:   [11, 12, 22, 25, 34, 64, 90]
```

```javascript
%%javascript
function bubbleSort(arr) {
    let n = arr.length;
    for (let i = 0; i < n; i++) {
        for (let j = 0; j < n - i - 1; j++) {
            if (arr[j] > arr[j + 1]) {
                // Swap elements
                let temp = arr[j];
                arr[j] = arr[j + 1];
                arr[j + 1] = temp;
            }
        }
    }
    return arr;
}

// Example usage in JavaScript
let javascript_arr = [64, 34, 25, 12, 22, 11, 90];
console.log(`Original JavaScript array: ${javascript_arr}`);
let sorted_javascript_arr = bubbleSort([...javascript_arr]); // Use spread operator to create a copy
console.log(`Sorted JavaScript array:   ${sorted_javascript_arr}`);
```

```
Original JavaScript array: 64,34,25,12,22,11,90
Sorted JavaScript array:   11,12,22,25,34,64,90
```

Explanation:

The AI translated an algorithm between two programming languages while maintaining the same logic. Differences in syntax, memory handling, and library support were observed. Both versions were tested to ensure correctness. This task highlights challenges and considerations when working across multiple programming languages.

Conclusion:

In this lab, we explored how AI can assist in translating code between different programming languages. It helped us understand differences in syntax, structure, and implementation across languages while maintaining the same logic. Overall, the lab showed how AI can simplify code conversion and improve learning across multiple programming environments.